鲜境 · 实验报告

实验四：AI 辅助企业级生鲜商城实践

独立品牌 · 仿盒马生鲜选购体验 · 实际持久化业务闭环

版本 1.0 | 编制日期 2026-10-06

依据实验任务书与教材第九章；记录本次实际实现及测试范围。

提交人姓名 / 学号 / 专业：_____

项目与提交信息

项目题目：【项目实战】企业级应用——鲜境生鲜商城系统。

姓名：_____ 学号：_____ 专业：_____

提交日期：_____ 小组成员与实际分工：_____

编程工具：Codex、Node.js 24、npm workspaces、TypeScript、Docker Desktop、Playwright。

大模型：GPT-6 系列（本会话系统标识；具体服务端构建号未公开）。

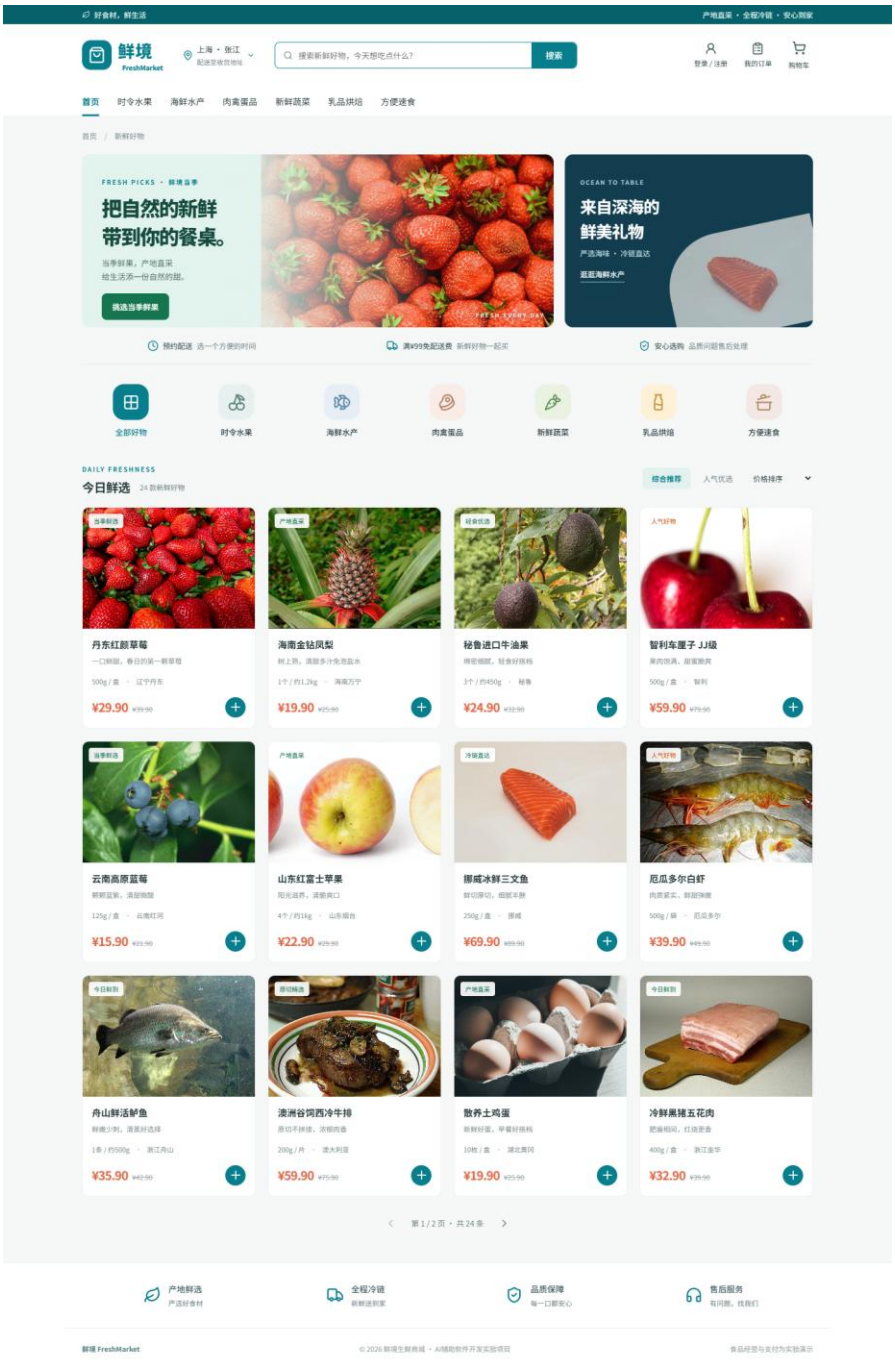
一、实验目的

根据教材第九章社区超市商城思路，完成企业级商城从需求、选型、数据建模、编码、测试到部署与工程化交付的完整实践。以生鲜购物为业务主题，验证购物闭环、角色职责、库存及资金沙箱一致性，并留下可复现证据。

二、实验步骤展示

2.1 立项与需求分析

系统名为“鲜境 FreshMarket”，面向城市家庭、通勤顾客和生鲜运营团队。采用独立品牌，借鉴盒马的分类选购、规格产地展示、冷链与到家体验。6 类 24 款食材；游客及 6 种账号角色；完整用例、流程、领域模型、ER、权限矩阵、API 与可验证指标见需求文档。



生鲜购物界面 · 完整原图：screenshots/01-home-desktop.png

2.2 技术选型与环境搭建

React 与 Fastify 为独立前后端工程，TypeScript 统一开发。Docker 实际启动 Nginx、API、PostgreSQL、Redis、RabbitMQ、Prometheus、Grafana 共 7 个容器。核心业务模块分层，配置中心由环境变量承担；模块化单体规模无需服务注册发现。精确依赖见锁文件，替代方案与风险见设计文档、ADR。

鲜境 FreshMarket · 实际运行环境

七个容器，真实连接的业务链路。

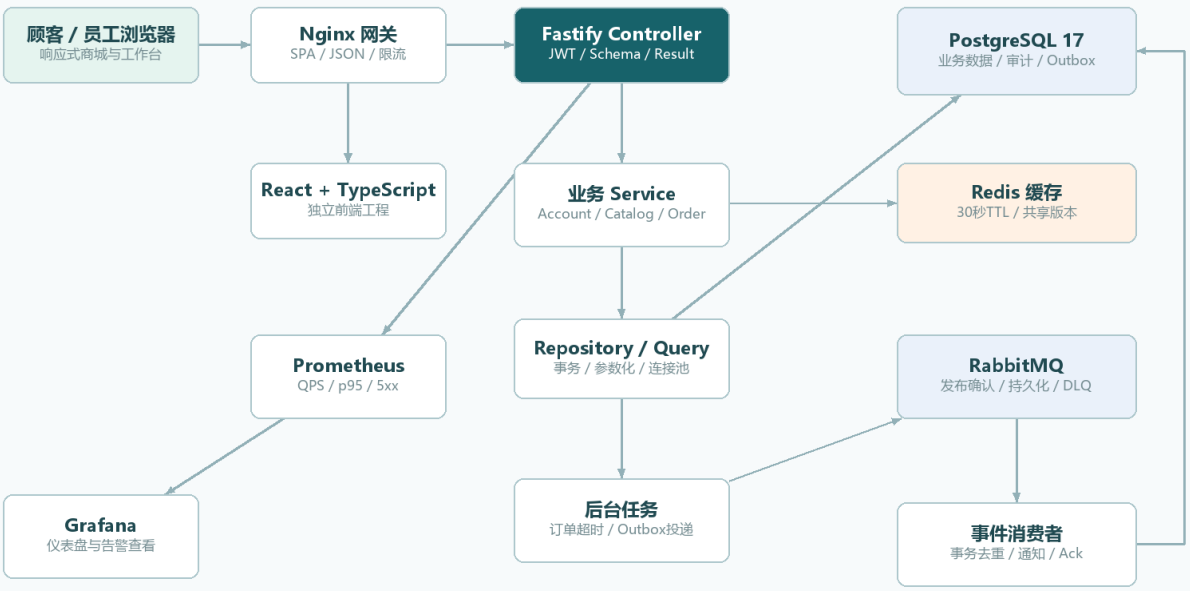
数据库	缓存 / 消息	监控采集
PostgreSQL	**Redis + RabbitMQ**	**Prometheus UP = 1**
事务与28项隔离测试	8项实物组件检查通过	实际指标查询结果

服务	镜像	运行状态	健康
api	freshmarket-api:local	running	healthy
grafana	grafana/grafana:13.2.3	running	运行中
postgres	postgres:17-alpine	running	healthy
prometheus	prom/prometheus:v3.13.4	running	运行中
rabbitmq	rabbitmq:4.3.6-management-alpine	running	healthy
redis	redis:7.4-alpine	running	healthy
web	freshmarket-web:local	running	healthy

2026-10-06T08:09:29.262Z · Node.js 24 / React 19 / Fastify 5 / TypeScript 5.9 · 原始证据位于 docs/evidence

实际部署环境 · 完整原图：screenshots/07-runtime-stack.png

鲜境：部署与分层架构



FreshMarket · 根据实际代码和数据库结构绘制

实际分层架构 · 完整原图：diagrams/architecture.png

2.3 AI 编程过程与关键决策

原始 Prompt：按照所给文档的要求完成系统搭建，目标是仿盒马搭建企业级生鲜购物平台。

工具/模型、阶段输入、生成内容、采纳理由、测试结果与实际修正记录见 AI 过程记录。采用整数分金额、服务端价格、下单快照、事务条件扣库存、行锁和唯一约束。采用 Outbox 同事务事件及 RabbitMQ 确认/去重，记录认证、缓存、消息及部署决策。个人与团队人工修改、评审和分工需由提交人补录。

2.4 业务实现与企业级能力

浏览→搜索→详情→购物车→地址/配送时段→幂等下单→沙箱付款→仓库拣货/配送→顾客收货/评价→财务结算。待付款取消及 15 分钟过期释放库存；整单退款审核冻结履约和结算。已配送生鲜不回库存。接口 JWT、密码 scrypt 哈希、RBAC、所有权校验、参数化 SQL、输入校验、限流、审计及 traceId；OpenAPI、Docker、监控、CI 配置、备份/回滚脚本均已交付。

服务端 RBAC 越权拦截

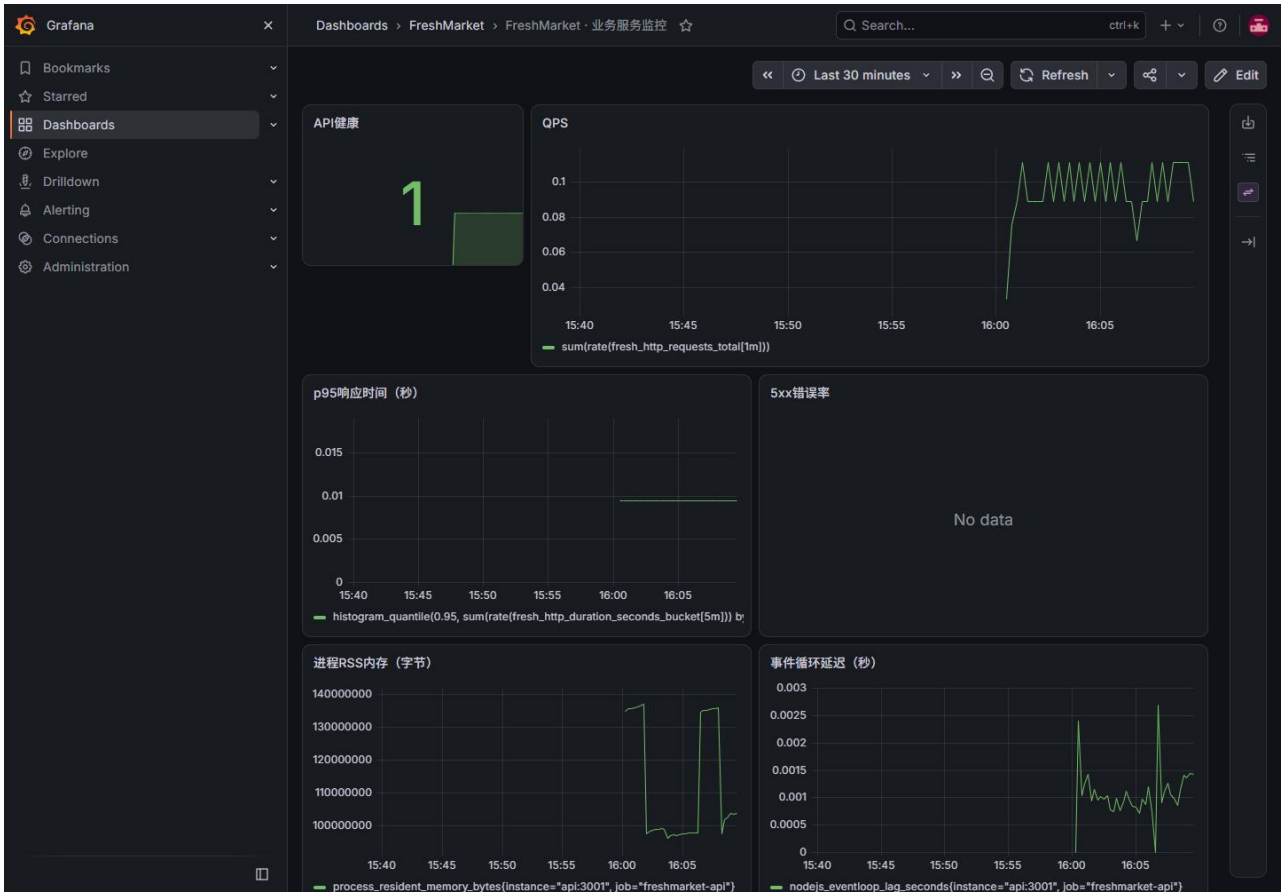
实际请求：warehouse → GET /api/admin/users

HTTP 403


```
{
  "code": "FORBIDDEN",
  "message": "当前账号无权执行此操作",
  "data": null,
  "traceId": "1800b799-67cb-41ae-bf0a-fc4d17f63959"
}
```


角色来自数据库；前端隐藏菜单之外，接口仍拒绝未授权请求。

真实越权拦截 · 完整原图：screenshots/10-rbac-denied.png



实际监控 · 完整原图：screenshots/08-grafana-monitor.png

2.5 测试验证

验证	最终实际结果
SQLite / PostgreSQL	各 28/28 通过
SQLite 覆盖率	行 87.02%，分支 80.63%
PostgreSQL 覆盖率	行 88.2%，分支 80.89%
浏览器 E2E	本地 5/5，完整 Docker5/5
实际 Redis/RabbitMQ 链路	8/8 通过
依赖审计	0 项已知告警
类型检查 / 生产构建	通过
本机商品接口短时测量	200 请求，并发 10，p95 17.34ms，错误率 0%

最后一项总时长约 0.168 秒，是本机短时突发测量；不代表持续容量或生产 SLA。CI 提供配置但未声明 GitHub 实际执行；恢复脚本未破坏性演练。

2.6 交付与演示

源码、SQL、部署脚本、Word 需求/设计/测试报告、实验报告、PPT、实际浏览器 WebM 录屏、原始测试证据及 SHA256 均交付。视频含购物闭环、真实 403、沙箱重复支付/退款、架构/Outbox 说明、监控、测试、AI 使用与复现；录屏字幕是讲解层。个人信息、真人讲解和手写签名由提交人完成。

三、开发亮点、问题及解决方案

难点	实际处理与证据
并发库存 / 订单幂等	锁用户与订单；条件扣减；8 次幂等提交、最后 3 件争抢测试
付款、取消、退款和结算竞争	状态机、行锁与唯一约束；重复操作和互斥测试
业务提交与消息投递可靠性	同事务 Outbox；ConfirmChannel；持久化消费去重；8 项实物组件验证
扫描教材无法直接读取	渲染指定章节页面，视觉读取实际 PDF150–156 页
Windows 子进程 / Docker 管道权限	使用执行工具权限流程；实际启动 Docker 并继续验证
E2E 登录定位歧义	限定 header 按钮，保留失败原因并回归
Swagger 关联依赖告警	升级 Swagger UI 6.1.1；重新审计及双数据库回归

四、小结

4.1 技术收获与体会

本项目的实践结论是：商城设计需围绕业务不变量建立数据约束，金额与库存由服务端负责；前端菜单权限必须与接口权限结合；事务内远程消息发布存在可靠性缺口，应采用持久化 Outbox 与消费去重；功能截图应与真实测试、部署和日志共同证明实现。AI 生成代码需经类型检查、并发与异常测试、依赖审计和真实组件验证后采纳。

个人或团队学习体会（提交人补充）：_____

4.2 今后改进方向

接入真实支付签名回调与退款渠道、实际配送和地理围栏、批次溯源与保质期管理；完善部分退款、已结算逆向账务、优惠券及多门店；验收集群限流、HTTPS、故障注入、长时压测、数据库高可用与备份恢复演练。当前实验功能边界在需求/设计/测试报告均有注明。

小组成员手写电子签名

成员姓名：_____ 实际分工：_____ 手写签名：_____

成员姓名：_____ 实际分工：_____ 手写签名：_____

以上信息须由小组据实填写，不由生成工具代签。